# Ders 20 Çalışma Özeti

## Ders 20 Çalışma Özeti

### Genel Konular

- Böl ve yönet (divide and conquer) yaklaşımı: böl (divide), yönet (conquer), birleştir (combine).
- Klasik sıralama algoritmalarının neden $O(n^2)$ olduğu ve bölmenin sağladığı kazanç.
- Azalt ve yönet (decrease and conquer) örneği olarak ikili arama (binary search).
- Özyineleme (recursion) hatırlatması: faktöriyel örneği, durma koşulu (base case) ve özyinelemeli adım (recursive case).
- İki sıralı dizinin birleştirilmesi (merge) ve karmaşıklığı.
- Birleştirerek sıralama (merge sort): mantığı, `mergeSort` ve `merge` kodları, adım adım izleme.
- Merge sort karmaşıklığı: yineleme bağıntısı (recurrence relation) ve geriye doğru yerine koyma (backward substitution).
- Hızlı sıralama (quick sort): pivot, bölümleme (partition) ve özyinelemeli yapı.

### Hocanın Özellikle Vurguladığı Kısımlar

- Algoritma tasarlarken basit düşünün. Aynı sonuca ulaşan iki çözümden basit olan tercih edilmelidir (Occam'ın usturası, Occam's razor).
- Böl ve yönet yaklaşımında bölündükten sonra bir parça üzerinde çalışırken diğer parçaya hiç bakılmaz. Kazanç buradan gelir.
- Böl ve yönet algoritmalarını çözmenin en iyi yolu özyinelemedir. Döngüyle de yazılabilir ama çok zahmetlidir.
- Gerekmeyen yerde özyineleme kullanmayın. Her fonksiyon çağrısının ek yükü (overhead) vardır. Faktöriyel gibi problemler döngüyle daha verimli çözülür.
- Özyinelemeli fonksiyonda durma koşulu tüm durumları karşılamalıdır. Karşılamazsa fonksiyon kendini çağırmaya devam eder ve yığın taşması (stack overflow) olur.
- Sıralı iki dizi birleştirilirken uç uca ekleyip sonra sıralamak doğru değildir. Dizilerin sıralı olmasından yararlanılmalıdır.
- Merge sort'taki "bölme" fiziksel değildir. Yalnız hangi indis aralığına bakılacağı belirlenir.
- Merge sort'un "hilesi" yedek dizi kullanmasıdır. Diğer sıralama algoritmaları aynı dizi üzerinde çalışır.
- Merge sort kararlıdır (stable). En iyi ve en kötü durumu aynıdır, her zaman $O(n \log n)$ çalışır.
- Quick sort'ta pivot gereksiz yere oynatılmaz. Önce diğer elemanlar yerleştirilir, pivot en sonda bir kez yer değiştirir.
- Problemi anlamak için önce elle 2-3 basit örnek, sonra 2 karmaşık örnek çözün. Girişte ne alınacağına ve çıkışta ne verileceğine kendiniz karar verebilmelisiniz.

### Kısa Tekrar Notları

- Böl ve yönet üç adımdır:
  - Böl: problemi daha küçük parçalara ayır.
  - Yönet: her parçayı diğerlerine karıştırmadan kendi içinde çöz.
  - Birleştir: parçaların çözümlerini birleştirerek bütün sonucu elde et.
- Selection, insertion ve bubble sort her adımda dizinin tamamıyla ilgilenir. Bu yüzden en kötü durumda $O(n^2)$'dir.

- İkili arama sıralı dizide yarıyı eler ve elenen yarıyla bir daha ilgilenmez. Hoca bunun böl ve yönet değil, azalt ve yönet olduğunu söyledi. Karmaşıklığı $O(\log n)$'dir.
- Özyinelemeli fonksiyonun iki parçası vardır: durma koşulu ve özyinelemeli çağrı.
- Faktöriyel: $n! = n \cdot (n-1)!$, $1! = 1$.
- Uzunlukları $n_A$ ve $n_B$ olan iki sıralı dizinin birleştirilmesi $O(n_A + n_B)$, yani $O(n)$'dir.
- Merge sort: diziyi tek elemanlı parçalara kadar böl. Tek elemanlı parça kendi içinde sıralıdır. Sonra parçaları birleştirerek yukarı çık.
- Merge sort'ta durma koşulu `p < r`'dir. `p == r` ise parçada tek eleman kalmıştır.
- Merge sort yineleme bağıntısı: $T(1) = c$, $n > 1$ için $T(n) = 2T(n/2) + n$. Çözüm $O(n \log_2 n)$.
- Merge sort'un tek dezavantajı ek bellek kullanmasıdır. Son birleştirmede dizinin boyu kadar yedek dizi gerekir.
- Quick sort: bir pivot seç (genelde ilk eleman). Pivottan küçükleri sola, büyük veya eşitleri sağa topla. Pivotu araya yerleştir. Sol ve sağ parçalar için aynı işlemi özyinelemeli tekrarla.
- Quick sort'ta dizi zaten sıralıysa ve pivot ilk elemansa karmaşıklık $O(n^2)$'ye çıkar.

## Detaylı Açıklamalar

### Böl ve Yönet Fikri

- Hoca bölmenin faydasını ilk elemanı 4 olan küçük bir diziyle gösterdi.
- 4'ten küçükler sola, büyükler sağa yerleştirildi. Sonuç: `2 1 3 | 4 | 7 15 11 5`.
- Artık 4 sıralı dizideki yerindedir.
- `2 1 3` sıralanırken sağ tarafa bakmaya gerek yoktur. `7 15 11 5` sıralanırken sol tarafa bakmaya gerek yoktur.
- Aynı işlem her parça için tekrarlanır. Örneğin `2 1 3` için 2 yerine yerleşir, `7 15 11 5` için 7 yerine yerleşir.
- Bir milyon elemanlı dizide pivot 200 bininci sıraya düşerse, solundaki 199.999 eleman sıralanırken sağdaki 800.000 elemana hiç bakılmaz.
- Bu aslında quick sort'un mantığıdır.
- Aynı işlem her parçada tekrarlandığı için çözüm doğal olarak özyinelemelidir.

### İkili Arama ve Azalt ve Yönet

- Sırasız dizide bir eleman aramak için dizinin tamamına bakılır (doğrusal arama).
- Sıralı dizide aranan değer ortadaki elemandan büyükse sol yarıya hiç bakılmaz.
- Her adımda yarısı elendiği için karmaşıklık $O(\log n)$'dir.
- Elenen yarıyla bir daha işlem yapılmadığı için hoca bunu azalt ve yönet (decrease and conquer) olarak adlandırdı. Bu türlerin ayrıntısı Algoritma Analizi dersinde görülecek.

### Özyineleme Hatırlatması: Faktöriyel

- Hoca faktöriyeli yalnız özyinelemeyi anlatmak için kullandı. Gerçekte döngüyle yazılmalıdır.
- Algoritma:
  - `n == 1` ise 1 döndür (durma koşulu).
  - Değilse `n * faktoriyel(n - 1)` döndür (özyinelemeli adım).
- $n < 1$ değerleri ana programda engellenmelidir.
- $F(3)$ izlemesi:
  - $F(3) = 3 \cdot F(2)$ çağrılır.
  - $F(2) = 2 \cdot F(1)$ çağrılır.
  - $F(1) = 1$ döner. Burada çağrılar durur.
  - Geri dönerken $F(2) = 2 \cdot 1 = 2$, $F(3) = 3 \cdot 2 = 6$ hesaplanır. Ana programa 6 döner.
- Fonksiyon çağrıları yığın (stack) bölgesini kullanır. Durma koşulu eksikse bu bölge dolar ve program "stack overflow" hatası verir.

### İki Sıralı Dizinin Birleştirilmesi (Merge)

- Örnek: `A = {2, 7, 9, 11, 15}`, `B = {8, 17, 19}`. Sonuç 8 elemanlı olur.

- Adımlar:
  - 2 ile 8 karşılaştırılır. 2 küçüktür ve yazılır. 2, 8'den küçükse 17 ve 19'dan da küçüktür.
  - 7 ile 8: 7 yazılır.
  - 8 ile 9: 8 yazılır.
  - 9 ile 17: 9 yazılır. 11 ile 17: 11 yazılır. 15 ile 17: 15 yazılır.
  - A dizisi bitti. B'nin kalan elemanları (17, 19) olduğu gibi sona eklenir.
- Sonuç: `2 7 8 9 11 15 17 19`.
- Hangi dizinin önce bittiğinin eleman sayısıyla ilgisi yoktur. Hoca bunu göstermek için kısa diziyi sona kalacak şekilde seçti.
- Merge bir sıralama değildir. İki bilgiyi birleştirme işlemidir. Ödevdeki iki grafın birleştirilmesi de bir merge işlemiydi.

### Merge Sort Mantığı

- Soru: Merge, sırasız tek bir diziyi sıralamak için kullanılabilir mi?
- Örnek dizi: `7 5 2 4 1 6 3 0`.
- `7 5 2 4` ve `1 6 3 0` parçaları sıralı değildir. Birleştirilemez.
- Bölmeye devam edilir. Tek elemana inildiğinde her parça kendi içinde sıralıdır.
- Sonra yukarı doğru birleştirilir:

```
Bölme:      7 5 2 4 1 6 3 0
            7 5 2 4 | 1 6 3 0
            7 5 | 2 4 | 1 6 | 3 0
            7 | 5 | 2 | 4 | 1 | 6 | 3 | 0
Birleştirme:
            5 7 | 2 4 | 1 6 | 0 3
            2 4 5 7 | 0 1 3 6
            0 1 2 3 4 5 6 7
```

- Son birleştirme: 0 ile 2 → 0, 1 ile 2 → 1, 3 ile 2 → 2, 3 ile 4 → 3, 6 ile 4 → 4, 6 ile 5 → 5, 6 ile 7 → 6, kalan 7 taşınır.
- Genel olarak `A[0..n-1]` dizisi `A[0..n/2-1]` ve `A[n/2..n-1]` diye ikiye ayrılır.

### Merge Sort ve Merge Kodu

Tahtadaki `mergeSort` (ekrandan, 01:45:50):

```
void mergesort(int A[], int p, int r)
{
    if (p < r)
    {
        q = (p + r) / 2;
        mergesort(A, p, q);
        mergesort(A, q + 1, r);
        merge(A, p, q, r);
    }
}
```

- `p` bakılan bölgenin başını, `r` sonunu gösterir. İlk çağrıda `p = 0`, `r = n - 1`'dir.
- `merge(A, p, q, r)`: `A[p..q]` ve `A[q+1..r]` sıralı parçalarını birleştirir.
- `merge` içinde yedek bir `B` dizisi kullanılır. Eleman sayısı `n = r - p + 1`'dir. Örneğin 3'ten 5'e kadar 3 eleman vardır.
- İki parça aynı A dizisi üzerindedir. `i` birinci parçada (`p`'den `q`'ya), `j` ikinci parçada (`q+1`'den `r`'ye) ilerler.
- Küçük olan `B`'ye yazılır. Döngü bitince bitmeyen parçanın kalanı `B`'ye taşınır.
- Son olarak `B`'deki sıralı bilgi `A[p..r]` bölgesine geri kopyalanır. `B` serbest bırakılır.

Aşağıdaki kod tahtadaki `merge` fonksiyonudur. Tahtadaki bir hata düzeltilmiş ve `malloc` / `free` satırları tamamlanmıştır (bkz. Ders sonrası notlar). Kod ajan tarafından küçük bir `main` ile `gcc -Wall` kullanılarak derlendi. Ders örneği `7 5 2 4 1 6 3 0` ile denendi ve `0 1 2 3 4 5 6 7` çıktısı alındı.

```c
void merge(int A[], int p, int q, int r)
{
    int n, i, j, k;
    int *B;
    n = r - p + 1;
    B = (int *) malloc(n * sizeof(int));
    if (B == NULL)
        exit(1);
    i = p; j = q + 1; k = 0;
    while ((i <= q) && (j <= r))
    {
        if (A[i] <= A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
    while (i <= q) B[k++] = A[i++];
    while (j <= r) B[k++] = A[j++];
    for (i = p, k = 0; i <= r; i++, k++)
        A[i] = B[k];
    free(B);
}
```

- `if (A[i] <= A[j])` koşulundaki eşitlik, eşit elemanlarda birinci parçadakini önce yazar. Merge sort'u kararlı yapan budur.
- Hoca bitmeyen parçayı taşımak için iki ayrı `while` yazdı. Bu, bir kontrol fazla yapar ama `if-else` ile yazmaktan daha sadedir.

### Merge Sort'un Adım Adım İzlenmesi

`7 5 2 4 1 6 3 0` için çağrı sırası:

```
mergesort(0,7)  q=3
  mergesort(0,3)  q=1
    mergesort(0,1)  q=0
      mergesort(0,0)  tek eleman, döner
      mergesort(1,1)  tek eleman, döner
      merge(0,0,1)    -> 5 7
    mergesort(2,3)  q=2
      mergesort(2,2), mergesort(3,3)
      merge(2,2,3)    -> 2 4
    merge(0,1,3)      -> 2 4 5 7
  mergesort(4,7)  q=5
    mergesort(4,5)  q=4
      mergesort(4,4), mergesort(5,5)
      merge(4,4,5)    -> 1 6
    mergesort(6,7)  q=6
      mergesort(6,6), mergesort(7,7)
      merge(6,6,7)    -> 0 3
    merge(4,5,7)      -> 0 1 3 6
  merge(0,3,7)        -> 0 1 2 3 4 5 6 7
```

### Neden Yedek Dizi Gerekiyor?

- `2 4 5 7 0 1 3 6` aynı A dizisindedir.
- 0, 2'den küçüktür ve başa gelmelidir. Ama 0 başa alınıp 2 onun yerine konursa ikinci parçanın sırası bozulur.
- Aynı dizi üzerinde birleştirme tasarımı çok karmaşıklaşır. Bu yüzden birleştirme yedek dizide yapılır.
- Yedek dizinin boyutu bakılan bölgenin boyutuna bağlıdır. Ama en son birleştirmede dizinin tamamı kadar yer gerekir.

**Merge Sort Karmaşıklığı**

- Kolaylık için $n$ ikinin kuvveti kabul edilir.
- `mergesort` kendini $n/2$ boyutunda iki kez çağırır ve bir kez $O(n)$'lik `merge` yapar.

Yineleme bağıntısı (ekrandan, 02:08:20):

$$T(n) = \begin{cases} c & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

Geriye doğru yerine koyma:

- $T(n) = 2T(n/2) + n$
- $= 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n$
- $= 4[2T(n/8) + n/4] + 2n = 8T(n/8) + 3n$
- Genel hâli: $T(n) = 2^i\, T(n/2^i) + i \cdot n$
- Durma $T(1)$'de olur. $n/2^i = 1$ için $n = 2^i$, yani $i = \log_2 n$.
- $T(n) = n \cdot 1 + n \log_2 n$.
- Büyük O gösteriminde en yüksek terim alınır: $O(n \log_2 n)$.
- $n$ ikinin kuvveti olmasa da sonuç yaklaşık aynıdır.
- Hoca, yineleme bağıntılarının diğer çözüm yollarının Algoritma Analizi dersinde görüleceğini söyledi.

**Quick Sort**

- En çok kullanılan sıralama algoritmalarındandır. Böl ve yönet yaklaşımının en etkileyici örneklerinden biridir.
- Hoca, küçük dizilerde özyinelemenin gereksiz yük getirdiğini söyledi. Bu yüzden kütüphanelerde küçük parçalar insertion sort ile sıralanır.
- Pivot genelde dizinin ilk elemanı seçilir. Başka pivot seçimleri sonraki derste konuşulacak.
- Derste öğrencilerden üç fikir geldi. Hoca üçünü de doğru buldu:
  - Pivotu her küçük elemanla yer değiştirerek ilerletmek.
  - Pivottan büyük bir eleman bulununca hemen sondaki elemanla yer değiştirmek.
  - İki işaretçi (`i` baştan, `j` sondan) kullanmak. Yer değiştirmeyi en aza indirdiği için tercih edilen yol budur.

Bölümleme adımları (dizi `5 3 1 9 8 2 4 7`, pivot 5, ekrandan 02:29:45):

- `i` soldan ilerler. Pivottan küçük elemanlar (3, 1) yerinde kalır. 9'da durur.
- `j` sağdan gelir. 7 büyüktür, geçilir. 4 küçüktür, durur. 9 ile 4 yer değiştirir: `5 3 1 4 8 2 9 7`.
- `i` 8'de durur. `j` 2'de durur. 8 ile 2 yer değiştirir: `5 3 1 4 2 8 9 7`.
- `i` ile `j` birbirini geçti. Bakılacak eleman kalmadı.
- Pivot son küçük elemanla (2) yer değiştirir: `2 3 1 4 | 5 | 8 9 7`. Pivot 4. göze yerleşti ($s = 4$).

Özyineleme ağacı:

```
(l=0, r=7) s=4
├── (l=0, r=3)  [2 3 1 4], pivot 2 -> 1 2 3 4, s=1
│   ├── (l=0, r=0)  tek eleman
│   └── (l=2, r=3)  [3 4], pivot 3, s=2
│       ├── (l=2, r=1)  boş, işlem yok
│       └── (l=3, r=3)  tek eleman
└── (l=5, r=7)  [8 9 7], pivot 8 -> 7 8 9, s=6
    ├── (l=5, r=5)  tek eleman
    └── (l=7, r=7)  tek eleman
Sonuç: 1 2 3 4 5 7 8 9
```

- Bölümlemeden sonra `A[0..s-1]` elemanları `A[s]`'den küçüktür, ama kendi aralarında sıralı değildir. `A[s+1..n-1]` elemanları `A[s]`'den büyük veya eşittir.

Tahtadaki sözde kod (ekrandan, 02:37:40):

```
QuickSort(A[l..r])        // ilk çağrı: l = 0, r = n-1
    if (l < r)
        s <- partition(A[l..r])   // pivotun yeni adresi
        QuickSort(A[l..s-1])
        QuickSort(A[s+1..r])
```

- `l == r` ise bakılacak tek eleman kalmıştır. İşlem yapılmaz. Durma noktası merge sort'takiyle aynı mantıktadır.
- `partition` fonksiyonunun kodu ve pivot seçimi bir sonraki derste ayrıntılı ele alınacak.

**Ders sonrası notlar**

- Tahtadaki `merge` kodunda başlangıç ataması `i = k = p;` yazıldı. `B` dizisi `n = r - p + 1` elemanlıdır ve 0'dan başlar. `k` de 0'dan başlamalıdır (`i = p; k = 0;`). Aksi hâlde `p > 0` olduğunda `B` dizisinin dışına yazılır. Geri kopyalama döngüsünde zaten `k = 0` kullanılmıştır. Ayrıca tahtadaki `mergesort` kodunda `q` değişkeni tanımlanmamıştır (`int q;` gerekir).
- Hoca merge sort'un von Neumann tarafından 1935'te bulunduğunu söyledi. Doğrusu 1945'tir.
- Bir yerde heap sort da böl ve yönet algoritmaları arasında sayıldı. Heap sort $O(n \log n)$'dir ama böl ve yönet algoritması değildir. Yığın (heap) veri yapısını kullanır.
- Hoca Java kütüphanesinin quick sort kullandığını söyledi. Daha doğrusu: Java'da `Arrays.sort` temel tipler (`int`, `double` vb.) için iki pivotlu quick sort (Dual-Pivot Quicksort) kullanır ve küçük parçalarda insertion sort'a geçer. Nesne dizilerinde ise merge sort tabanlı TimSort kullanılır.
- Merge sort'un "her zaman $O(n \log n)$" olması, klasik yukarıdan aşağı (top-down) sürümü içindir. Ek bellek ihtiyacı $O(n)$'dir.
- Altyazıda Whisper bazı terimleri bozmuştur: "nörd sort / mörç / WordSort" = merge sort, "Divayden konkur / Tüvalden konkur" = divide and conquer, "Digrizen Komupuru" = decrease and conquer, "off comes razor" = Occam's razor.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.